

Construcción de un Servicio Base para un Sistema Distribuido

Microservicio **veterinaria-mascota-ms** — CRUD de Mascotas y Especies con Spring Boot, PostgreSQL, Flyway y Swagger

Basado en la guía S01_Construccion_Servicio_Base · 262dist/pagatu

Repositorio	Tarea-1
Autor / commits	JoaHasss
Stack	Java 21 · Spring Boot 4.0.7 · PostgreSQL 16
Fecha del informe	27 de agosto de 2026
Último commit	c9b50eb

1. Objetivo y alcance

Este informe documenta la construcción del microservicio `veterinaria-mascota-ms`, desarrollado siguiendo la guía de la Sesión 1 del curso *Desarrollo de Aplicaciones Distribuidas* ("Construcción de un servicio base para un sistema distribuido"). El ejercicio original de la guía propone un microservicio de catálogo (`categorías` / `productos`); en este repositorio el mismo patrón arquitectónico se aplicó a un dominio propio de **veterinaria**: gestión de **especies** y **mascotas**, manteniendo idéntica capa de arquitectura, validaciones, manejo de errores, trazabilidad y documentación.

El objetivo de la sesión, tal como lo define la guía, es "*construir e implementar un microservicio sin estado (stateless) con API REST, persistencia en PostgreSQL, validación de entradas, documentación Swagger, verificación de salud vía Actuator y ejecución reproducible en desarrollo, con soporte horizontal mediante múltiples instancias.*" Todos estos puntos fueron implementados y verificados en este repositorio, con evidencia capturada en las secciones 8 a 11.

Nota sobre el historial de commits. El repositorio conserva dos commits: el primero (`ffb671d`) construyó el servicio base siguiendo literalmente el enunciado de la guía (dominio de menú de restaurante); el segundo (`c9b50eb`) refactorizó el mismo esqueleto hacia el dominio final de veterinaria (mascotas/especies) y añadió puerto dinámico para facilitar el escalamiento horizontal. Este informe describe el estado final del código, que es el que corre y se evalúa.

1.1 Stack tecnológico empleado

Componente	Tecnología	Rol en el proyecto
Lenguaje / runtime	Java 21 LTS	Lenguaje base del microservicio
Framework	Spring Boot 4.0.7	Web MVC, inyección de dependencias, autoconfiguración
Persistencia	Spring Data JPA + Hibernate	Mapeo objeto-relacional de <code>Mascota</code> y <code>Especie</code>
Base de datos	PostgreSQL 16 (contenedor Docker)	Almacenamiento relacional
Migraciones	Flyway	Control de versiones del esquema (<code>V1__create_menu_tables.sql</code>)
Documentación API	SpringDoc OpenAPI / Swagger UI	Exploración y prueba interactiva de endpoints
Observabilidad	Spring Boot Actuator	Endpoint de salud <code>/actuator/health</code>
Validación	Jakarta Bean Validation	<code>@NotBlank</code> , <code>@NotNull</code> , <code>@Size</code> , <code>@DecimalMin</code> en los DTO de entrada
Build	Maven Wrapper (<code>mvnw</code>)	Compilación reproducible sin instalar Maven
Reducción de boilerplate	Lombok	Getters/Setters/Builder en entidades y DTO
Infra local	Docker Compose (<code>compose-dev.yml</code>)	PostgreSQL de desarrollo aislado

2. Arquitectura por capas

Se implementó la arquitectura en capas propuesta por la guía, aplicada a los dos recursos del dominio (`Especie` como entidad padre, `Mascota` como entidad hija con relación `@ManyToOne`):



Cada petición HTTP entra por el `CorrelationIdFilter` , que asigna (o respeta) un identificador de traza; el `Controller` valida el payload con `@Valid` y delega en el `Service` , que aplica las reglas de negocio (por ejemplo, verificar que la especie exista antes de crear una mascota) y usa el `Mapper` para convertir entre `Entity` y DTO antes de persistir vía `Repository` . Los errores de negocio y de validación se centralizan en `GlobalExceptionHandler` , que traduce excepciones a respuestas JSON con código HTTP correcto.

2.1 Estructura de paquetes del proyecto

```
src/main/java/pe/edu/upeu/veterinaria/
├─ VeterinariaApplication.java      (bootstrap @SpringBootApplication)
├─ controller/
│   ├── EspecieController.java      (/api/v1/especies)
│   └─ MascotaController.java       (/api/v1/mascotas)
├─ service/
│   ├── EspecieService.java
│   └─ MascotaService.java
├─ mapper/
│   ├── EspecieMapper.java
│   └─ MascotaMapper.java
├─ entity/
│   ├── Especie.java
│   └─ Mascota.java
├─ repository/
│   ├── EspecieRepository.java
│   └─ MascotaRepository.java       (findAllConEspecie -> JOIN FETCH)
├─ dto/
│   ├── EspecieRequest.java / EspecieResponse.java
│   └─ MascotaRequest.java / MascotaResponse.java
├─ exception/
│   ├── ResourceNotFoundException.java
│   └─ GlobalExceptionHandler.java
└─ filter/
    └─ CorrelationIdFilter.java

src/main/resources/
├─ application.yml                  (nombre de la app + perfil activo "dev")
├─ application-dev.yml              (datasource, flyway, jpa, swagger, actuator)
├─ logback-spring.xml               (patrón de log con traceId)
└─ db/migration/
    └─ V1__create_menu_tables.sql    (tablas especies y mascotas)
```

3. Modelo de datos y migraciones (Flyway)

Siguiendo el patrón de la guía, el esquema se versiona con Flyway y Hibernate se configura únicamente en modo de **validación** (`ddl-auto:validate`), de forma que la estructura de tablas la controla siempre la migración SQL y no el ORM.

```
-- V1__create_menu_tables.sql
CREATE TABLE IF NOT EXISTS especies (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY,
  nombre VARCHAR(100) NOT NULL,
  descripcion VARCHAR(255),
  PRIMARY KEY (id)
);

CREATE TABLE IF NOT EXISTS mascotas (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY,
  nombre VARCHAR(100) NOT NULL,
  descripcion VARCHAR(255),
  peso NUMERIC(10,2) NOT NULL,
  activo BOOLEAN NOT NULL DEFAULT true,
  id_especie BIGINT NOT NULL REFERENCES especies(id),
  PRIMARY KEY (id)
);
```

Detalle observado: el archivo de migración conserva el nombre histórico `V1__create_menu_tables.sql` (heredado del commit inicial sobre el dominio de restaurante). El *contenido* ya corresponde 100% al dominio de veterinaria (tablas `especies` y `mascotas`); es solo el nombre de archivo el que no se actualizó en el refactor. No afecta la ejecución porque Flyway versiona por prefijo `V1`, no por el resto del nombre.

`mascotas.id_especie` referencia a `especies.id`, modelando la relación 1:N entre especie y mascota que en la capa de entidades se representa como `@ManyToOne(fetch = FetchType.LAZY)`.

4. Implementación por capas

4.1 Entidades JPA

Dos entidades con Lombok (`@Getter/@Setter/@Builder`) y mapeo explícito de columnas:

```
// Mascota.java
@Entity @Table(name = "mascotas")
@Getter @Setter @Builder @NoArgsConstructor @AllArgsConstructor
public class Mascota {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false, length = 100) private String nombre;
    @Column(length = 255) private String descripcion;
    @Column(nullable = false, precision = 10, scale = 2) private BigDecimal peso;
    @Column(nullable = false) private Boolean activo;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "id_especie", nullable = false)
    private Especie especie;
}
```

4.2 DTO y validación de entrada

La API nunca expone las entidades JPA directamente: los `Request` llevan las anotaciones de validación de Jakarta Bean Validation y los `Response` exponen solo los campos necesarios para el cliente.

```
// MascotaRequest.java
@NotBlank @Size(max = 100) private String nombre;
@Size(max = 255) private String descripcion;
@NotNull @DecimalMin("0.0") private BigDecimal peso;
@NotNull private Boolean activo;
@NotNull private Long especieId;
```

4.3 Mapper y prevención de N+1

El requisito de la guía de "evitar N+1 en la relación producto-categoría" se resolvió de dos formas combinadas: (a) el repositorio expone una consulta con `JOIN FETCH` para el listado, y (b) el `MascotaResponse` incluye el objeto `EspecieResponse` completo (no solo el id), construido por el mapper en una sola pasada.

```
// MascotaRepository.java
@Query("SELECT m FROM Mascota m JOIN FETCH m.especie")
List<Mascota> findAllConEspecie();

// MascotaMapper.java (fragmento)
public MascotaResponse toResponse(Mascota m) {
    return MascotaResponse.builder()
        .id(m.getId()).nombre(m.getNombre()).peso(m.getPeso())
        .especie(especieMapper.toResponse(m.getEspecie())) // objeto anidado completo
        .build();
}
```

4.4 Servicio: reglas de negocio

`MascotaService` valida la existencia de la especie referenciada antes de crear o actualizar una mascota, lanzando `ResourceNotFoundException` si no existe:

```
public MascotaResponse crear(MascotaRequest request) {
    Mascota mascota = mascotaMapper.toEntity(request);
    mascota.setEspecie(buscarEspecieOFallar(request.getEspecieId()));
    return mascotaMapper.toResponse(mascotaRepository.save(mascota));
}

private Especie buscarEspecieOFallar(Long id) {
    return especieRepository.findById(id)
}
```

```
.orElseThrow(() -> new ResourceNotFoundException("Especie no encontrada: " + id));  
}
```

4.5 Controlador REST

Endpoints mapeados con versión de API en la URL (`/api/v1/...`), tal como recomienda la guía para compatibilidad futura:

Método	Ruta	Código éxito	Descripción
GET	<code>/api/v1/especies</code>	200	Lista todas las especies
GET	<code>/api/v1/especies/{id}</code>	200 / 404	Obtiene una especie por id
POST	<code>/api/v1/especies</code>	201	Crea una especie (valida payload)
PUT	<code>/api/v1/especies/{id}</code>	200 / 404	Actualiza una especie
DELETE	<code>/api/v1/especies/{id}</code>	204 / 404	Elimina una especie
GET	<code>/api/v1/mascotas</code>	200	Lista mascotas con especie anidada (JOIN FETCH)
GET	<code>/api/v1/mascotas/{id}</code>	200 / 404	Obtiene una mascota por id
POST	<code>/api/v1/mascotas</code>	201 / 400 / 404	Crea una mascota (valida payload + existencia de especie)
PUT	<code>/api/v1/mascotas/{id}</code>	200 / 404	Actualiza una mascota
DELETE	<code>/api/v1/mascotas/{id}</code>	204 / 404	Elimina una mascota

5. Manejo de errores y trazabilidad distribuida

5.1 Respuestas de error centralizadas

`GlobalExceptionHandler` (`@RestControllerAdvice`) intercepta dos tipos de excepción y devuelve siempre el mismo formato JSON (`timestamp` , `status` , `error` , `message`):

```
@ExceptionHandler(ResourceNotFoundException.class)
// -> 404 NOT_FOUND, message = ex.getMessage()

@ExceptionHandler(MethodArgumentNotValidException.class)
// -> 400 BAD_REQUEST cuando @Valid rechaza el payload
```

5.2 Correlation ID (X-Trace-ID)

`CorrelationIdFilter` extiende `OncePerRequestFilter` : si la petición ya trae el header `X-Trace-ID` lo respeta, si no genera un `UUID` nuevo; en ambos casos lo inyecta en el `MDC` de SLF4J (para que aparezca en cada línea de log) y lo devuelve en la respuesta, habilitando trazabilidad extremo a extremo entre microservicios.

```
String traceId = request.getHeader("X-Trace-ID");
if (traceId == null || traceId.isBlank()) traceId = UUID.randomUUID().toString();
MDC.put("traceId", traceId);
response.setHeader("X-Trace-ID", traceId);
```

`logback-spring.xml` incluye `%X{traceId}` en el patrón de log, escribiendo simultáneamente a consola y a `logs/veterinaria.log` con rotación diaria.

6. Configuración de entorno y escalamiento horizontal

Configuración dividida en `application.yml` (nombre de app y perfil activo) y `application-dev.yml` (datasource, Flyway, JPA, Swagger, Actuador). El puerto del servidor se parametrizó como `server.port: ${PORT:0}`, es decir: si no se define la variable de entorno `PORT`, Spring Boot toma un puerto libre aleatorio; si se define, usa ese puerto exacto. Esto permite levantar N instancias del mismo jar en la misma máquina sin colisión de puertos, todas apuntando a la misma base de datos PostgreSQL — el patrón de escalamiento horizontal sin balanceador que pide la guía.

```
# application-dev.yml (fragmento)
server:
  port: ${PORT:0}
spring:
  datasource:
    url: jdbc:postgresql://localhost:15432/veterinaria_mascota_db
  flyway:
    enabled: true
  jpa:
    hibernate:
      ddl-auto: validate # el esquema lo controla Flyway, no Hibernate
```

```
# compose-dev.yml – PostgreSQL de desarrollo aislado
postgres-mascota-dev:
  image: postgres:16-alpine
  environment:
    POSTGRES_DB: veterinaria_mascota_db
    POSTGRES_USER: veterinaria
  ports: ["15432:5432"]
```

Para esta evidencia se levantaron **dos instancias reales** del mismo artefacto (`veterinaria-mascota-ms-0.0.1-SNAPSHOT.jar`) con `PORT=8080` y `PORT=8081` respectivamente, ambas compartiendo la misma base de datos. Las capturas de la sección 8 muestran `/actuator/health` respondiendo `UP` en los dos puertos simultáneamente.

7. Procedimiento de ejecución (reproducido para este informe)

Pasos ejecutados en PowerShell/bash para levantar y validar el servicio, en el mismo orden que indica la guía:

```
$ docker compose -f compose-dev.yml up -d
Container veterinaria-postgres-mascota-dev Running

$ ./mvnw -DskipTests clean package
BUILD SUCCESS -> target/veterinaria-mascota-ms-0.0.1-SNAPSHOT.jar

$ PORT=8080 java -jar target/veterinaria-mascota-ms-0.0.1-SNAPSHOT.jar
$ PORT=8081 java -jar target/veterinaria-mascota-ms-0.0.1-SNAPSHOT.jar
... Tomcat started on port 8080
... Tomcat started on port 8081
... Started VeterinariaApplication in ~23s (Flyway aplicó V1 sobre la BD vacía)
```

8. Evidencia: documentación y salud del servicio

Capturas tomadas directamente contra el servicio corriendo en `localhost` mientras se elaboraba este informe.

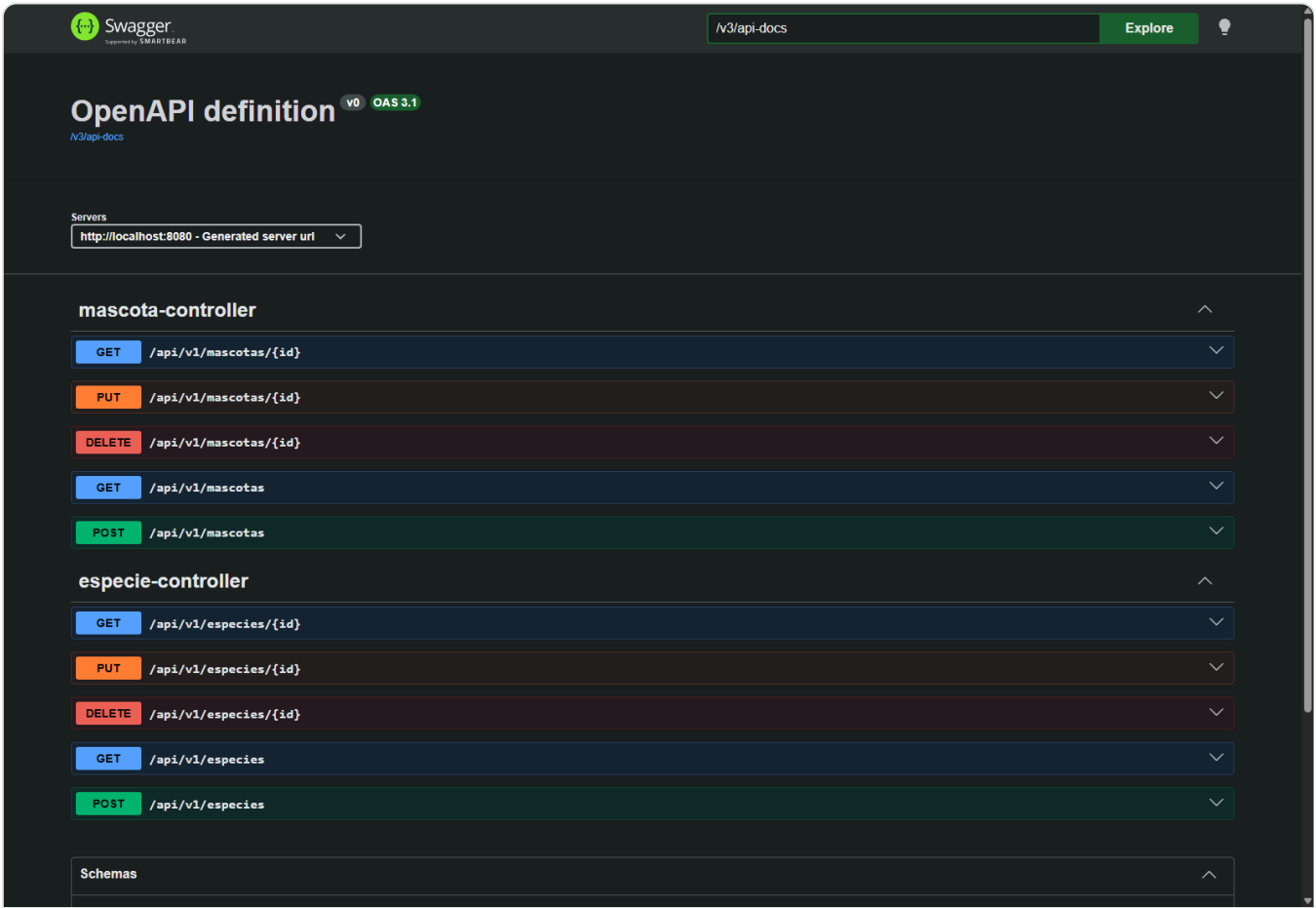


Figura 1 — Swagger UI (`/swagger-ui/index.html`) en el puerto 8080, mostrando los 10 endpoints generados automáticamente para `mascota-controller` y `especie-controller` a partir de las anotaciones REST.

9. Evidencia: pruebas funcionales del CRUD (línea de comandos)

Se ejecutó el ciclo completo de pruebas contra `http://localhost:8080` con `curl`, replicando el checklist de verificación de la guía (creación, lectura, actualización, borrado, validación 400 y recurso inexistente 404).

9.1 Alta de especies y mascotas

```
$ curl -X POST /api/v1/especies -d '{"nombre":"Canino","descripcion":"Perros domesticos"}'
201 {"id":3,"nombre":"Canino","descripcion":"Perros domesticos"}

$ curl -X POST /api/v1/mascotas -d '{"nombre":"Firulais","peso":18.5,"activo":true,"especieId":3,...}'
201 {"id":3,"nombre":"Firulais","peso":18.5,"activo":true,
    "especie":{"id":3,"nombre":"Canino","descripcion":"Perros domesticos"}}
-- nótese el objeto "especie" completo anidado en la respuesta (sección 4.3) --
```

9.2 Listado, actualización y borrado

```
$ curl -X PUT /api/v1/mascotas/4 -d '{"nombre":"Michi","peso":4.6,...}'
200 {"id":4,"nombre":"Michi","peso":4.6,"descripcion":"Gato adoptado, esterilizado",...}

$ curl -X DELETE /api/v1/mascotas/4
HTTP 204 (No Content - borrado correcto)
```

9.3 Manejo de errores (400 y 404)

```
$ curl /api/v1/mascotas/4 # tras el DELETE anterior
HTTP 404 {"error":"Not Found","message":"Mascota no encontrada: 4","status":404}

$ curl -X POST /api/v1/mascotas -d '{"nombre":"","peso":-5}' # payload invalido
HTTP 400 {"error":"Bad Request","message":"Error de validación en los datos enviados","status":400}

$ curl /api/v1/mascotas/999 # id inexistente
HTTP 404 {"error":"Not Found","message":"Mascota no encontrada: 999","status":404}
```

9.4 Trazabilidad con X-Trace-ID

```
$ curl -D - /api/v1/especies | grep X-Trace-ID
X-Trace-ID: 04ade88d-fde7-4388-baa7-f6c680eb7099 # generado automaticamente

$ curl -H "X-Trace-ID: mi-id-personalizado-123" -D - /api/v1/especies | grep X-Trace-ID
X-Trace-ID: mi-id-personalizado-123 # el filtro respeta el id del cliente
```

10. Verificación contra los criterios mínimos de la guía

- ✓ Proyecto estructurado como microservicio independiente (Maven, `mvnw`, un solo `pom.xml`).
- ✓ Dos entidades relacionadas (`Especie` 1—N `Mascota`) con mapeo JPA correcto.
- ✓ CRUD completo (GET/POST/PUT/DELETE) para ambos recursos, verificado por línea de comandos (sección 9).
- ✓ Persistencia en PostgreSQL vía migraciones Flyway, Hibernate en modo `validate`.
- ✓ Validación de entradas que rechaza datos inválidos con HTTP 400 (sección 9.3).
- ✓ Manejo de excepciones centralizado con códigos HTTP correctos (404 / 400).
- ✓ Documentación Swagger accesible y fiel a los DTO reales (sección 8, figuras 1-2).
- ✓ Endpoint de salud de Actuator funcional (sección 8, figuras 3-4).
- ✓ Múltiples instancias ejecutándose simultáneamente sobre la misma BD (puerto dinámico, sección 6 y 8).
- △ README con instrucciones operativas: **no presente** en el repositorio actual — pendiente de agregar.
- △ Control de versiones con tags (`s01-servicio-base-p1`, `s01-servicio-base-p2`): **el repositorio no tiene tags creados** aún, solo los dos commits descritos en la sección 1.

11. Respuestas a las preguntas de sustentación de la guía

1. ¿Por qué usar DTO en lugar de exponer las entidades directamente?

Porque la entidad JPA representa el modelo de persistencia (columnas, relaciones lazy, anotaciones de Hibernate) y no necesariamente el contrato que debe ver el cliente. Exponerla directamente acoplaría la API a la base de datos: cualquier cambio de esquema rompería el contrato público, se podrían filtrar campos internos y las relaciones `@ManyToOne(LAZY)` como `Mascota.especie` lanzarían `LazyInitializationException` al serializar fuera de la sesión de Hibernate. Los DTO (`MascotaRequest` / `MascotaResponse`) desacoplan ambos mundos y permiten anotar validación solo donde corresponde (entrada).

2. ¿Cómo habilita trazabilidad el filtro de correlación?

`CorrelationIdFilter` asegura que todo request tenga un `X-Trace-ID` (propio o heredado del llamador) disponible en el `MDC` de logging durante toda la vida de la petición, y lo devuelve en la respuesta. Así, si este microservicio llamara a otro (por ejemplo `auth-ms` en sesiones futuras), reenviando el mismo header, todas las líneas de log de ambos servicios para una misma transacción distribuida quedarían correlacionadas por el mismo id, sin necesidad de un sistema de tracing externo.

3. ¿Qué evita el problema N+1 en la relación mascota-especie?

El listado usa `@Query("SELECT m FROM Mascota m JOIN FETCH m.especie")` en `MascotaRepository`, que trae mascota y especie en una sola consulta SQL (un `JOIN`) en lugar de una consulta por mascota para resolver su especie perezosa. El mapper solo transforma en memoria lo que ya vino cargado, así que listar N mascotas cuesta 1 consulta, no N+1.

4. ¿Por qué aplicar las migraciones antes de la validación de JPA?

Con `ddl-auto: validate`, Hibernate no crea ni modifica tablas: solo compara el modelo de entidades contra el esquema existente y falla si no coincide. Por eso Flyway debe ejecutarse primero (crea las tablas reales desde `V1__create_menu_tables.sql`); si el orden se invirtiera, Hibernate encontraría una base de datos vacía y el arranque fallaría. Este orden también evita que dos fuentes de verdad (Hibernate autogenerando DDL y Flyway) compitan por el esquema en un entorno con múltiples instancias corriendo a la vez.

5. ¿Cómo funciona el escalamiento horizontal sin balanceador?

Al no guardar estado en memoria (sin sesiones HTTP con datos de negocio, toda la persistencia vive en PostgreSQL) y al parametrizar el puerto (`server.port: ${PORT:0}`), el mismo artefacto `.jar` puede levantarse N veces en paralelo —como se demostró en 8080 y 8081— y cualquiera de las instancias puede atender cualquier request indistintamente, porque todas comparten la misma fuente de verdad (la base de datos). En este ejercicio no hay balanceador todavía: cada instancia se prueba de forma independiente: el balanceo real llega en sesiones posteriores del curso (API Gateway, sesión 4).

12. Conclusiones

El microservicio `veterinaria-mascota-ms` implementa, de forma verificable y con evidencia reproducible, todos los elementos centrales de la guía S1: arquitectura en capas, persistencia versionada con Flyway, validación de entrada, manejo centralizado de errores, trazabilidad por correlation id, documentación Swagger y ejecución en múltiples instancias sin estado compartido. Los dos puntos pendientes frente al checklist de la guía son de forma, no de arquitectura: falta un `README.md` operativo y no se han creado los tags de Git que marcan los hitos de la sesión (`s01-servicio-base-p1` tras las migraciones, `s01-servicio-base-p2` tras el CRUD completo). Ambos son ajustes rápidos y no requieren tocar código.

Informe generado a partir de la inspección del código fuente en `Tarea-1` (commit `c9b50eb`) y de una ejecución real del servicio (PostgreSQL vía Docker Compose + dos instancias Java en los puertos 8080/8081) el 27 de agosto de 2026.

Guía de referencia: 262dist.github.io/pagatu/sesiones/S01_Construccion_Servicio_Base/